

DocSearch

Multi-Tenant Document Search Engine

Architecture & Production Readiness Report

Project	DocSearch — In-Memory Multi-Tenant Search API
Stack	Spring Boot 4.1 · Java 17 · Lombok
Author	Abhishek Dhiman
Date	June 2026
Version	0.1.0-SNAPSHOT

1. Architecture Design

1.1 System Overview

DocSearch is a RESTful document search engine built on Spring Boot 4.1. It provides full multi-tenant isolation, inverted-index search at the word and trigram (n=3) levels, and Levenshtein edit-distance fuzzy matching — all backed by in-memory data structures for prototype speed.

1.2 Component Map

Layer	Class	Responsibility
API	TenantController	Tenant CRUD — POST/GET/DELETE /api/v1/tenants
API	DocumentController	Document CRUD + search — /api/v1/documents, /api/v1/search
API	GeneralController	Health check — GET /health-check
Service	TenantService	Owns Tenant entities and per-tenant TenantIndex instances
Service	DocumentService	Indexes, retrieves, deletes docs; runs 3-pass search
Store	TenantIndex	Per-tenant: docs map, word inverted index, trigram index
Model	Tenant	id · name · createdAt
Model	Document	id · tenantId · title · content · createdAt
DTO	CreateDocumentRequest	title · content (POST body)
DTO	CreateTenantRequest	name (POST body)
DTO	SearchResult	document · score (search response)

1.3 Request Flow

Every inbound request follows this path:

- HTTP client sends request with **X-Tenant-Id** header (or **?tenant=** for search).
- **DocumentController** validates tenant existence via **TenantService.exists()** — returns 404 if unknown.
- **DocumentService** calls **TenantService.getIndex(tenantId)** to retrieve that tenant's isolated **TenantIndex**.
- All reads/writes operate exclusively on that **TenantIndex** — no shared state across tenants.
- Search returns a scored, ranked **List<SearchResult>**.

1.4 Inverted Index Design

Each **TenantIndex** maintains two separate ConcurrentHashMaps:

- **wordIndex**: token → Set<docId> — exact word lookup.
- **trigramIndex**: 3-char gram → Set<docId> — substring / fuzzy lookup.

Indexing — on every POST /documents:

```
tokenize("Spring Boot Guide") → ["spring", "boot", "guide"]
trigrams("spring")           → ["spr", "pri", "rin", "ing"]
wordIndex["spring"]           → {docId}
trigramIndex["spr"]           → {docId} (etc.)
```

1.5 Search Scoring (3-pass)

For each query token:

Pass	Condition	Points	Purpose
1	edit_distance(query, word) == 0	+10	Exact word match
2	edit_distance(query, word) <= maxD	+(10 - d*3)	Fuzzy word match
3	trigram overlap	+1 / gram	Partial / substring

The **maxEditDistance** threshold is read from **application.properties** (key: **search.max-edit-distance**, default 2) and injected via **@Value**. No restart is required if externalised via environment variable or config server.

2. Production Readiness Analysis

The prototype demonstrates the core algorithm and API contract. The table below maps each production concern to its current state and a recommended upgrade path.

Persistence	Not implemented
All data lives in JVM heap — lost on restart. Upgrade: add Spring Data JPA + PostgreSQL (documents table) and a dedicated search store such as Elasticsearch or OpenSearch.	
Authentication / AuthZ	Not implemented
No auth on any endpoint. Upgrade: Spring Security + JWT (stateless) or OAuth 2.0 with a tenant-scoped scope claim. The X-Tenant-Id header must be derived from the verified token, not trusted from the client.	
Input Validation	Partial
No @NotNull/@NotBlank constraints on DTOs. Upgrade: add jakarta.validation annotations + @Valid in controller params. Return 400 with structured error body.	
Error Handling	Partial
404 / 204 returned correctly; no global exception handler. Upgrade: @ControllerAdvice + ProblemDetail (RFC 7807) for consistent error envelope.	
API Rate Limiting	Not implemented
No throttling — a single tenant can saturate the JVM. Upgrade: Bucket4j or API gateway-level rate limiting per tenantId.	
Observability	Not implemented
No metrics, tracing, or structured logging. Upgrade: Micrometer + Prometheus for metrics, OpenTelemetry for traces, Logback JSON encoder for structured logs.	
Horizontal Scaling	Not implemented
In-memory state is not shared across pods. Upgrade: externalise indexes to Redis (inverted index as sorted sets) or Elasticsearch. Stateless Spring Boot instances behind a load balancer.	
Tenant Lifecycle	Partial
Tenant delete cascades to indexes but not via a transactional boundary. Upgrade: wrap in a distributed transaction or saga when using an external store.	
Search Quality	Partial
TF-IDF weighting not implemented; all word occurrences count equally. Upgrade: store term frequency at index time; multiply score by IDF.	
Pagination	Not implemented
Search returns all matching docs. Upgrade: add ?page=&size;= params; return Page with total count.	

Configuration Mgmt**Partial**

search.max-edit-distance in application.properties. Upgrade: Spring Cloud Config or environment-variable overrides for per-environment tuning.

CI / CD**Not implemented**

No pipeline defined. Upgrade: GitHub Actions — build + test on PR, Docker image push on merge to main, Helm chart deploy to Kubernetes.

Containerisation**Not implemented**

No Dockerfile. Upgrade: multi-stage Dockerfile (eclipse-temurin:17-jdk-alpine builder + JRE runtime layer). Expose port 8080, set JVM heap via -XX:MaxRAMPercentage.

Health / Readiness**Partial**

GET /health-check returns plain string. Upgrade: Spring Boot Actuator (/actuator/health with readiness + liveness probes for k8s).

Secret Management**Not implemented**

No secrets in use yet, but no vault integration either. Upgrade: HashiCorp Vault or AWS Secrets Manager; never commit credentials to source.

3. Experience Showcase

This section highlights the engineering decisions made during the prototype and what they demonstrate about system design capability.

Structural Tenant Isolation

Rather than filtering a shared store by a key prefix (error-prone), each tenant receives its own `TenantIndex` object — separate `ConcurrentHashMaps` for docs, word index, and trigram index. Cross-tenant data leakage becomes a compile-time impossibility: there is no shared collection to accidentally query without a tenant context.

Layered Fuzzy Search

Three complementary signals are combined into a single relevance score: (1) exact word match via the word inverted index, (2) edit-distance word matching using Levenshtein (iterative, $O(m*n)$ time, $O(n)$ space), and (3) raw trigram overlap for substring and partial matches. This mirrors the approach used in production search engines such as Elasticsearch's match query pipeline.

Configurable Edit Distance

The maximum Levenshtein distance is externalised to `application.properties` (`search.max-edit-distance`). This single line controls the fuzziness budget for the entire system — no recompilation needed. In a production system this would be per-tenant or per-index configurable.

Inverted Index with Trigram Pre-filter

Trigrams act as a cheap candidate filter before the more expensive Levenshtein computation. Only words that share at least one trigram with the query token are evaluated for edit distance, bounding the worst-case cost to $O(V * m * n)$ where V is the matching vocabulary fraction rather than the full vocabulary.

Score-ranked Responses

Search results surface the raw relevance score alongside the document. This makes the ranking decision transparent for debugging and allows clients to apply their own threshold (e.g. hide results with score < 5).

Clean Service Boundaries

`TenantService` owns tenant lifecycle and hands `TenantIndex` instances to `DocumentService`. `DocumentService` has no knowledge of HTTP. Controllers have no business logic — they translate HTTP concerns (headers, status codes) and delegate. This separation means the search algorithm can be unit-tested without Spring context.

Concurrent-safe Data Structures

`ConcurrentHashMap` is used throughout. `computeIfAbsent` / `computeIfPresent` are used for atomic index updates, avoiding lost-update races under concurrent indexing.

4. AI Tool Usage

Claude Code (Anthropic) was used as a pair-programmer throughout this project. The following describes what was AI-assisted and where human judgement drove decisions.

AI-Assisted Tasks

Scaffolding — Initial controller stubs and DTO classes were generated from a natural-language description of the API contract, saving boilerplate time.

Boilerplate — Levenshtein implementation, tokenizer, and trigram generator were generated and verified against known examples.

Dependency Diagnosis — The root cause of the original /health-check 500 error (wrong Maven artifact names: spring-boot-starter-webmvc instead of spring-boot-starter-web) was identified by the AI in one step.

Documentation — This PDF was authored with AI assistance — structure, content, and Python/ReportLab code.

Human-Led Decisions

Architecture Decisions — The choice to use per-tenant TenantIndex instances (structural isolation) over a shared map with prefix filtering was a deliberate design decision made by the engineer, not suggested by the AI.

Scoring Design — The three-pass scoring model (exact=10, edit-distance scaled, trigram=1) and the decision to expose score in the API response were human-driven.

Production Readiness Prioritisation — Identifying which gaps matter most for a real multi-tenant SaaS product (auth, persistence, rate limiting) reflects domain experience, not AI output.

Code Review — Every generated snippet was read, understood, and accepted or modified before being committed. The AI produced no code that was not reviewed.

AI tooling accelerated execution on well-defined tasks ("write a Levenshtein function", "generate a DTO") while leaving architectural thinking, trade-off analysis, and production readiness judgement to the engineer. The result is a prototype that reflects genuine engineering reasoning, not uncritical AI output.